Homework — 2.1.3 Thinking Procedurally

Name: ___ Date: _ ***Mark:*** ___ / 20

OCR H446 — set after the 2.1.3 lesson.

Part A — This week's topic (~14 marks)

1. A team is building an online cinema-ticket booking system. Using *thinking procedurally*, identify **three** sensible sub-procedures the problem could be decomposed into. **[3]** *(AO2)*

2. In that booking system the steps include: `selectSeats` → `takePayment` → `issueTicket`. Explain why `issueTicket` must come **after** `takePayment`, referring to the output of an earlier step. **[2]** *(AO2)*

3. A washing-machine control program runs: lock door → fill with water → heat water → wash → drain → spin.

(a) State **one** pair of these steps whose order is fixed, and explain the dependency. **[2]** *(AO2)*

(b) Suggest **one** step here that could in principle be done independently of another, briefly justifying why. **[1]** *(AO2)*

4. A program for an online quiz calls the same `gradeAnswer` sub-procedure for every question. Explain **two** advantages of writing this once as a sub-procedure rather than copying the grading code under each question. **[2]** *(AO2)*

5. A games company has four programmers building one large open-world game. Explain how **decomposition** lets them work effectively as a team. **[2]** *(AO2)*

6. A council wants software to process residents' parking-permit applications. Using *thinking procedurally*, decompose this into **named** sub-procedures in a sensible **order**, and for one pair of steps explain why the order is fixed (one step needs an earlier step's output). **[2]** *(AO2)*

Part B — Retrieval: keep it fresh (~6 marks)

7. *(2.1.2)* State what a **precondition** is, and give one example for a routine that performs a **binary search**. **[2]** *(AO1/AO2)*

8. *(1.2.2)* State, in order, **two** stages of compilation that come **before** code generation. **[2]** *(AO1)*

9. *(1.2.3)* State **one** advantage of the **Agile** development methodology over the traditional **Waterfall** model. **[2]** *(AO1)*